

Competitive Reverse-Engineering

Uncover the discovery behind an AI product

1. Overview

Pick one of three everyday AI products and reverse-engineer the discovery work that must have happened before it was built. This exercise trains you to think like a PM by uncovering the problems that features solve, not by building features.

Due Date

Submit by **3rd July**, before Session 9 on 4th July.

2. Pick Your Product

Choose one of these three everyday AI products:

1. LinkedIn Job Recommendations

AI-suggested job postings tailored to your profile, skills, and career history.

2. Amazon Rufus

AI shopping assistant that answers product questions, compares items, and helps with purchase decisions.

3. Gmail Smart Reply

AI-suggested short replies that appear below your inbox emails.

3. Your Tasks (90 minutes)

3.1 Map the user journey

Where, when, and how do users encounter this AI feature? What does a typical session look like?

Example: For Gmail Smart Reply, a user receives an email, opens inbox on mobile, sees three suggested replies below the message, taps one, and the email is sent in 2 seconds. The entire job is "respond quickly to low-stakes emails."

3.2 Identify the job-to-be-done

What does this feature enable that wasn't possible before? Why would a user choose this over doing it manually?

Example: Before Smart Reply, users had to type full responses. The job-to-be-done is "acknowledge an email without friction." Users choose it when they're busy, on mobile, or the email doesn't need a thoughtful reply.

3.3 Spot the trade-offs

Every AI feature makes trade-offs (speed vs. accuracy, personalization vs. privacy, simplicity vs. control). What trade-offs did this team make? What did they prioritize, and what did they sacrifice?

Example: Gmail Smart Reply prioritizes speed and simplicity over personalization. The suggested replies are generic ("Thanks for the update!") so they work for everyone, but they can't match your tone or context. The team sacrificed personality to gain universal usability. A trade-off worth it for quick emails, but not for nuanced conversations.

3.4 Spot the job they're NOT solving

What related problem does this feature ignore? What job did the team consciously leave unsolved, and why might that matter?

Example: Gmail Smart Reply doesn't help you write better emails, only reply faster. It doesn't solve "I need to sound more professional" or "I want to negotiate better terms." The team left the composition job unsolved, probably because it's harder to scale AI suggestions when tone, context, and stakes vary wildly. For quick acknowledgments, this is fine. For important emails, users still draft manually.

3.5 Find the adjacent opportunity

If this feature works well, what's the next problem users will face? What discovery question would uncover that next feature?

Example: After Gmail Smart Reply succeeds, users might realize "I spend even more time on drafting new emails from scratch." The adjacent opportunity is "Smart Compose" (AI suggests full email drafts, not just replies). The discovery question becomes: "Do users want AI help writing emails they initiate, not just replying to others?" Gmail and other teams likely asked this and found a real job-to-be-done.

4. What to Submit

Create a one-page PDF with:

- Product name and a one-sentence description of the AI feature.
- User journey (2 to 3 sentences or simple visual).
- Job-to-be-done (2 to 3 sentences).

- One paragraph: "Trade-offs: what they prioritized vs. sacrificed."
- One paragraph: "The job they're NOT solving: and why it matters."
- One paragraph: "The adjacent opportunity: and the discovery question it raises."

5. How to Share

Post your one-page PDF in the Discord channel. PDF submissions only.

6. Peer Feedback

Read 2 to 3 of your peers' submissions and leave freeform comments. What trade-offs stood out to you? Did they spot a job the product is ignoring? Do you see patterns in the adjacent opportunities they identified?

7. Why This Matters

Great products start with clear problem discovery, not feature ideas. By reverse-engineering a real product, you'll see how discovery choices shape what gets built and what gets left out. You'll internalize the questions PMs ask before they design: What job are we solving? What are we not solving? What's the next problem after this one? These habits will guide every PRD and product decision you make.

8. Grading

We'll spot-check a few submissions for depth and clarity. Focus on making your thinking visible. Your trade-off analysis and adjacent opportunity should reveal real PM reasoning, not surface observations.